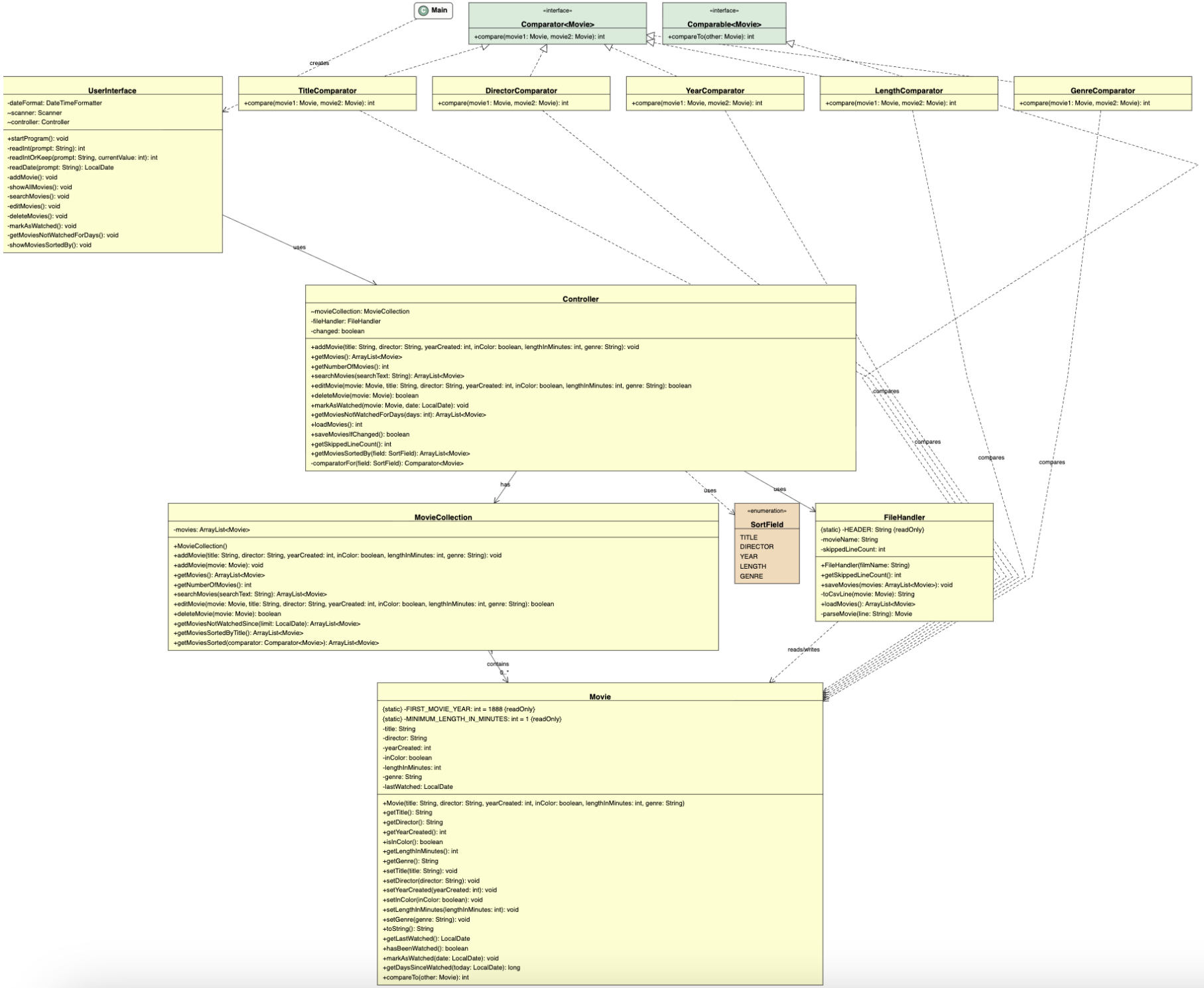

Main
«interface»
Comparator<Movie>
+compare(movie1: Movie, movie2: Movie): int
«interface»
Comparable<Movie>
+compareTo(other: Movie): int
creates
UserInterface
-dateFormat: DateTimeFormatter
~scanner: Scanner
~controller: Controller
+startProgram(): void
-readInt(prompt: String): int
-readIntOrKeep(prompt: String, currentValue: int): int
-readDate(prompt: String): LocalDate
-addMovie(): void
-showAllMovies(): void
-searchMovies(): void
-editMovies(): void
-deleteMovies(): void
-markAsWatched(): void
-getMoviesNotWatchedForDays(): void
-showMoviesSortedBy(): void
TitleComparator
+compare(movie1: Movie, movie2: Movie): int
DirectorComparator
+compare(movie1: Movie, movie2: Movie): int
YearComparator
+compare(movie1: Movie, movie2: Movie): int
LengthComparator
+compare(movie1: Movie, movie2: Movie): int
GenreComparator
+compare(movie1: Movie, movie2: Movie): int
uses
Controller
~movieCollection: MovieCollection
-fileHandler: FileHandler
-changed: boolean
+addMovie(title: String, director: String, yearCreated: int, inColor: boolean, lengthInMinutes: int, genre: String): void
+getMovies(): ArrayList<Movie>
+getNumberOfMovies(): int
+searchMovies(searchText: String): ArrayList<Movie>
+editMovie(movie: Movie, title: String, director: String, yearCreated: int, inColor: boolean, lengthInMinutes: int, genre: String): boolean
+deleteMovie(movie: Movie): boolean
+markAsWatched(movie: Movie, date: LocalDate): void
+getMoviesNotWatchedForDays(days: int): ArrayList<Movie>
+loadMovies(): int
+saveMoviesIfChanged(): boolean
+getSkippedLineCount(): int
+getMoviesSortedBy(field: SortField): ArrayList<Movie>
-comparatorFor(field: SortField): Comparator<Movie>
compares
compares
compares
compares
has
uses
uses
MovieCollection
-movies: ArrayList<Movie>
+MovieCollection()
+addMovie(title: String, director: String, yearCreated: int, inColor: boolean, lengthInMinutes: int, genre: String): void
+addMovie(movie: Movie): void
+getMovies(): ArrayList<Movie>
+getNumberOfMovies(): int
+searchMovies(searchText: String): ArrayList<Movie>
+editMovie(movie: Movie, title: String, director: String, yearCreated: int, inColor: boolean, lengthInMinutes: int, genre: String): boolean
+deleteMovie(movie: Movie): boolean
+getMoviesNotWatchedSince(limit: LocalDate): ArrayList<Movie>
+getMoviesSortedByTitle(): ArrayList<Movie>
+getMoviesSorted(comparator: Comparator<Movie>): ArrayList<Movie>
«enumeration»
SortField
TITLE
DIRECTOR
YEAR
LENGTH
GENRE
FileHandler
{static} -HEADER: String {readOnly}
-movieName: String
-skippedLineCount: int
+FileHandler(filmName: String)
+getSkippedLineCount(): int
+saveMovies(movies: ArrayList<Movie>): void
-toCsvLine(movie: Movie): String
+loadMovies(): ArrayList<Movie>
-parseMovie(line: String): Movie
reads/writes
contains
0..*
Movie
{static} -FIRST_MOVIE_YEAR: int = 1888 {readOnly}
{static} -MINIMUM_LENGTH_IN_MINUTES: int = 1 {readOnly}
-title: String
-director: String
-yearCreated: int
-inColor: boolean
-lengthInMinutes: int
-genre: String
-lastWatched: LocalDate
+Movie(title: String, director: String, yearCreated: int, inColor: boolean, lengthInMinutes: int, genre: String)
+getTitle(): String
+getDirector(): String
+getYearCreated(): int
+isInColor(): boolean
+getLengthInMinutes(): int
+getGenre(): String
+setTitle(title: String): void
+setDirector(director: String): void
+setYearCreated(yearCreated: int): void
+setInColor(inColor: boolean): void
+setLengthInMinutes(lengthInMinutes: int): void
+setGenre(genre: String): void
+toString(): String
+getLastWatched(): LocalDate
+hasBeenWatched(): boolean
+markAsWatched(date: LocalDate): void
+getDaysSinceWatched(today: LocalDate): long
+compareTo(other: Movie): int